

Parallel Programming, Spr. 2026, Lecture 15: Data Races, Solving Mutual Exclusion with Atomic Registers

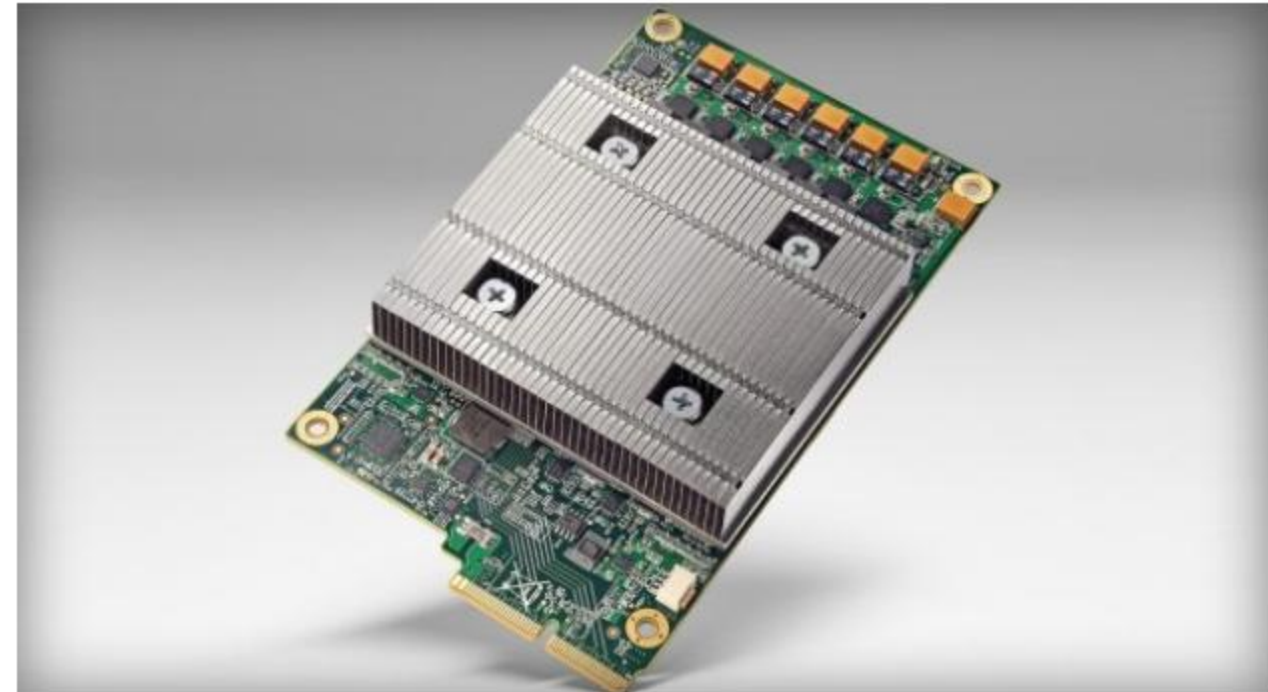
TORSTEN HOEFLER



Google's dedicated TensorFlow processor, or TPU, crushes Intel, Nvidia in inference workloads

By Joel Hruska on April 6, 2017 at 9:48 am | 23 Comments

2.2K
shares



Several years ago, Google began working on its own custom software for machine learning and artificial intelligence workloads, dubbed **TensorFlow**. Last year, the company announced that it had designed its own tensor processing unit (TPU), an ASIC designed for high throughput of low-precision arithmetic. Now, Google has released some performance data for their TPU and how it compares to Intel's Haswell CPUs and Nvidia's

In-Datcenter Performance Analysis of a Tensor Processing Unit™

Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley

Google's Cloud TPU v4 provides exaFLOPS-scale ML with industry-leading efficiency

April 6, 2023



DESIGNLINES | AI & BIG DATA DESIGNLINE

Stories Claiming Google 'Beat Nvidia' Greatly Overstated

By Sally Ward-Foxton 04.10.2023 0

Share Post [Share on Facebook](#) [Share on Twitter](#) [in](#)

←

Ads by Google

[Send feedback](#) [Why this ad? ▸](#)

Google researchers published a [paper](#) last week discussing the performance of its home-grown TPuv4 AI accelerator chips, which garnered a lot of attention. In the paper, the researchers compare the performance of their chips with market leader Nvidia's GPUs. This has been reported as "[Google reveals its newest A.I. supercomputer, says it beats Nvidia,](#)" according to CNBC, amongst [others](#).

In the paper, the researchers are not claiming the TPuv4 outpaces Nvidia's current-gen flagship AI accelerator, the H100. Instead, Google is comparing its TPuv4 to the previous-gen A100. The way this has been reported has prompted some industry-watchers to claim either that Google is "beating Nvidia," or that Google is somehow making unfair comparisons. Neither is correct.

(caches, out-of-order execution, multithreading, multiprocessing, prefetching, ...) that help average throughput more than guaranteed latency. The lack of such features helps explain why, despite having myriad MACs and a big memory, the TPU is relatively small and low power. We compare the TPU to a server-class Intel Haswell CPU and an Nvidia K80 GPU, which are contemporaries deployed in the same datacenters. Our workload, written in the high-level TensorFlow framework, uses production NN applications (MLPs, CNNs, and LSTMs) that represent 95% of our datacenters' NN inference demand. Despite low utilization for some applications, the TPU is on average about 15X - 30X faster than its contemporary GPU or CPU, with TOPS/Watt about 30X - 80X higher. Moreover, using the GPU's GDDR5 memory in the TPU would triple achieved TOPS and raise TOPS/Watt to nearly 70X the GPU and 200X the CPU.

Index terms—DNN, MLP, CNN, RNN, LSTM, neural network, domain-specific architecture, accelerator

What is left for us humans?

Brian Lamacraft
Jan 5 · 2 min read · Member-only

Chat GTP-4 Could Pass the Bar Exam

How Our Technology Evolves FAST

Source: <https://medium.com/>

AI chatbot's MBA exam pass poses test for business schools

ChatGPT earned a solid grade and outperformed some humans on a Wharton course

AI Passes U.S. Medical Licensing Exam

— Two papers show that large language models, including ChatGPT, can pass the USMLE

by Michael DePeau-Wilson, Enterprise & Investigative Writer, MedPage Today January 19, 2023

Source: <https://www.medpagetoday.com/>

ChatGPT Passes Google Coding Interview for Level 3 Engineer With \$183K Salary

'Amazingly, ChatGPT gets hired at L3 when interviewed for a coding position,' reads a Google document, but ChatGPT itself says it can't replicate human creativity and problem-solving skills.

by Emily Dreibelb Feb 01, 2023



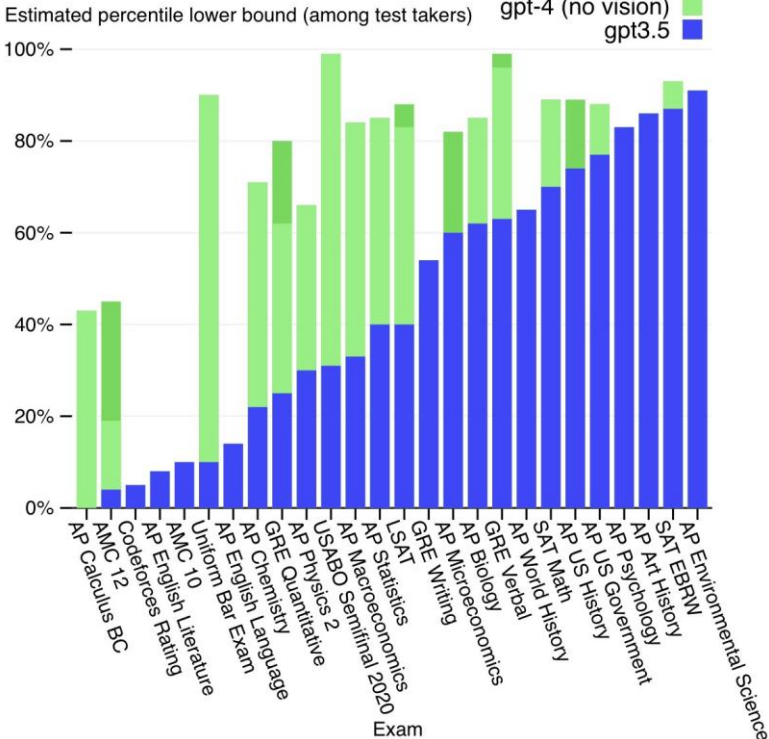
Bing Chat Outshines ChatGPT with Cutting-Edge "Theory of Mind" Capabilities

SHAUN Feb 13, 2023 Updated • Feb 13, 2023

Apps | 7

Oh, ChatGPT, if you ever develop a consciousness, please remember with joy all the fun you're having with humanity nowadays. Especially if you ever consider squeezing out our life force while being locked in a vat. As you surely know, ChatGPT has been on everybody's minds lately thanks to its incredible capabilities. You probably know about ChatGPT's ability to write a poem, develop software, and even write phishing emails for all those wannabe criminals out there.

Exam results (ordered by GPT-3.5 performance)



“Really the deciding factor [for the AI revolution] was the increase in compute power” (26:50)

“I think a lot of the credit for deep learning goes to [... others ...] and the people who made the computers go fast.” (27:00)

2018 ACM A.M. Turing Lecture

June 23, 2019

5:15pm MST



Geoffrey Hinton



Yann LeCun



FINANCIAL TIMES

Artificial intelligence

+ Add to myFT

The billion-dollar bet to reach human-level AI

OpenAI believes that huge computing power is key driver

In the race to build a machine with human-level intelligence, it seems, size really matters.

“We think the most benefits will go to whoever has the biggest computer,” said Greg Brockman, chairman and chief technology officer of OpenAI.

The San Francisco-based AI research group, set up four years ago by tech industry luminaries including Elon Musk, Peter Thiel and Reid Hoffman, has just thrown down a challenge to the rest of the AI world.

Richard Waters in San Francisco

AUGUST 3 2019



140



How do we “Make Computers go Fast”?

2021 Turing award – Jack Dongarra

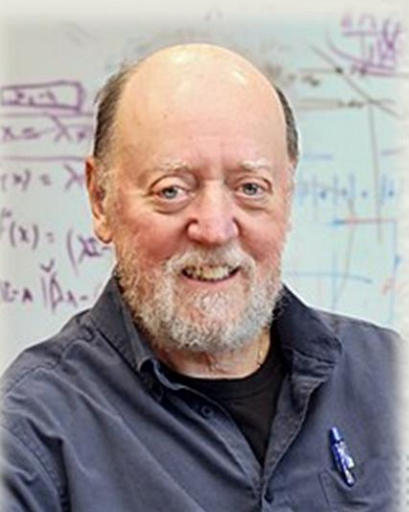
The T

Supercomputers
are very (>70%)
efficient at dense
linear algebra!

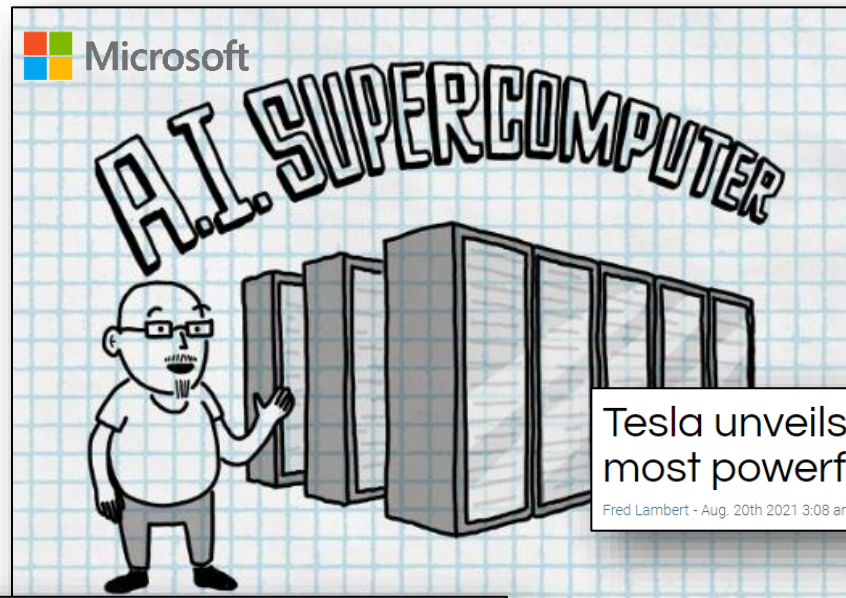
- HPC
- Distributed
- Accelerated
- Mixed precision
- Three computer revolutions
 - High performance computing
 - Deep learning
 - Edge & AI
- Algorithm / Software advances follow
 - And there is “plenty of room”

“for fundamental contributions to high-performance computing and the ongoing AI revolution - developed many of the core capabilities of modern supercomputers and defined key aspects of the algorithms for distributing AI models on them.”

ACM Prize in Computing (2024)
“Most prestigious CS award
for young researchers”



<https://www.youtube.com/watch?v=lsnRP9akCDk>



Meta has built an AI supercomputer it says will be world's fastest by end of 2022



Tesla unveils Dojo supercomputer: world's new most powerful AI training machine
Fred Lambert - Aug. 20th 2021 3:08 am PT @FredericLambert

... next generation of machine learning

By James Vincent | Jan 24, 2022, 12:00pm EST

... meanwhile in big industry ...

DOI:10.1145/3380307

Google's TPU supercomputers train deep neural networks 50x faster than general-purpose supercomputers running a high-performance computing benchmark.

BY NORMAN P. JOUPPI, DOE HYUN YOON, GEORGE KURIAN, SHENG LI, NISHANT PATIL, JAMES LAUDON, CLIFF YOUNG, AND DAVID PATTERSON

A Domain-Specific Supercomputer for Training Deep Neural Networks



ANNOUNCING THE WORLD'S FASTEST SUPERCOMPUTER FOR AI

20 Exaflops of AI

Powered by NVIDIA Grace CPU and Next Generation NVIDIA GPU

HPC and AI for Scientific and Commercial Apps
Advance Weather, Climate, and Material Science

 **CSCS**
Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

 **Hewlett Packard Enterprise**

 **NVIDIA**


ALPS
CSCS ETH zürich

More on our youtube channel ... or X, LinkendIn, or BlueSky

<https://www.youtube.com/@spcl>

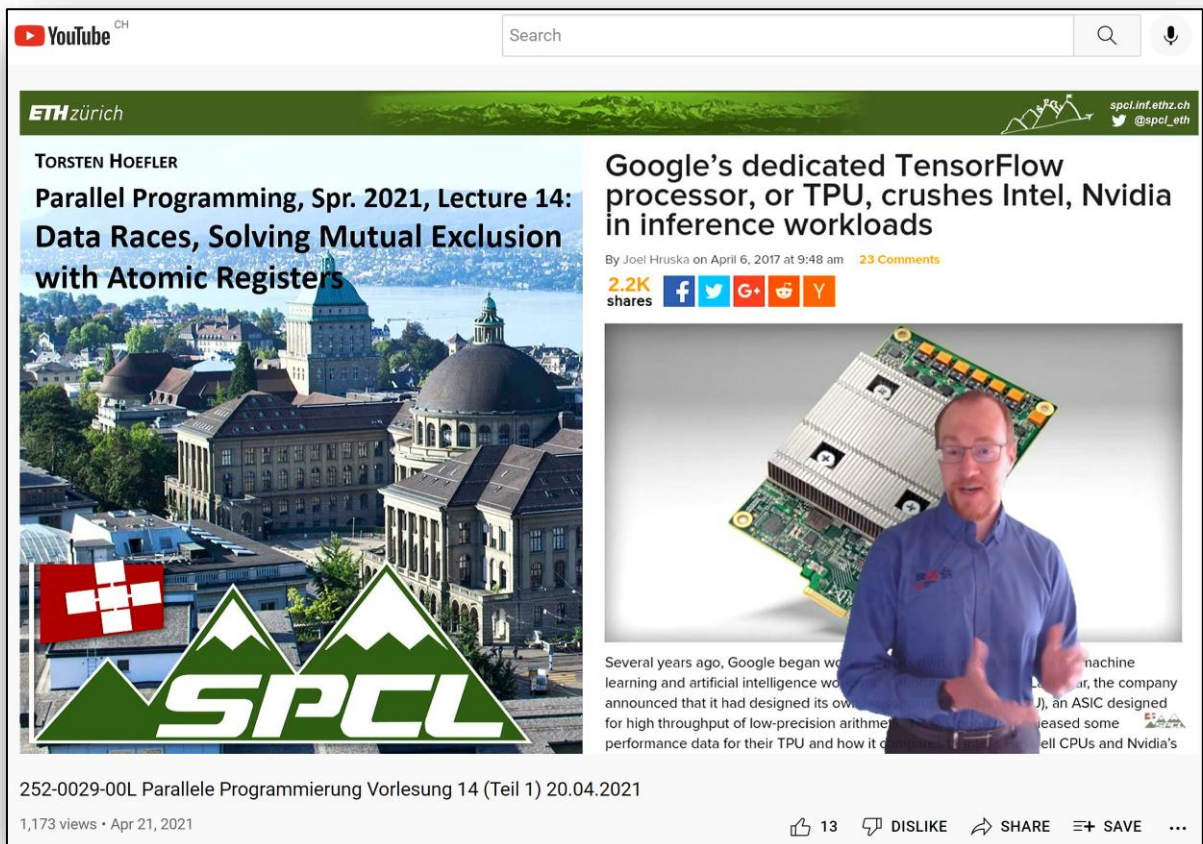


T. HOEFLER
From Large Language Models to Reasoning Language Models
Three Eras in The Age of Computation.
with contributions by the whole SPCL deep learning team (M. Besta, J. Barth, E. Schreiber, T. Ben-Nun, S. Li, and many others), Microsoft Azure (M. Heddes, J. Belk, S. Scott, D. Goel, M. Castro) and collaborators (D. Alistarh and others)
Presentation at ADIA Lab Seminar, Abu Dhabi, UAE on January 28 2025

From Large Language Models to Reasoning Language Models - Three Eras in The Age of Computation.

Scalable Parallel Computing Lab, SPCL @ ETH ...
3.71K subscribers

579 likes, 1 comment, 1 share, 1 download, 1 clip



TORSTEN HOEFLER
Parallel Programming, Spr. 2021, Lecture 14: Data Races, Solving Mutual Exclusion with Atomic Registers

Google's dedicated TensorFlow processor, or TPU, crushes Intel, Nvidia in inference workloads

By Joel Hruska on April 6, 2017 at 9:48 am 23 Comments
2.2K shares

Several years ago, Google began working on a dedicated machine learning and artificial intelligence workload accelerator. Last year, the company announced that it had designed its own TPU v2, an ASIC designed for high throughput of low-precision arithmetic. Google released some performance data for their TPU and how it compares to Intel CPUs and Nvidia's GPUs.

252-0029-00L Parallele Programmierung Vorlesung 14 (Teil 1) 20.04.2021

1,173 views · Apr 21, 2021

13 likes, 0 dislikes, 1 share, 1 save

Learning goals for today

So far:

- Programming with locks and critical sections
- Key guidelines and trade-offs
- Bad interleavings (high level races)

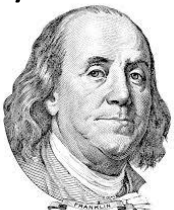
Now:

- The unfortunate reality of parallel programming in practice – memory models
- **Why you must avoid data races** (= low level races / memory reorderings)
- Implementation of a Mutex with Atomic Registers

Dekker's algorithm

Peterson's algorithm

- Context: remember you will likely not implement locks (you will use functions provided by the programming language!)
YET: you will learn important principles by “doing” – and watching your (my) mistakes carefully



“Tell me and I forget, teach me and I may remember, involve me and I learn.”

Motivation

```
class C {
    private int x = 0;
    private int y = 0;
```

Thread 1

`x = 1;` (A)

`y = 1;` (B)

}

Thread 2

`int a = y;` (C)

`int b = x;` (D)

`assert(b >= a);`

}

}

Can this fail?

There is no *interleaving* of f and g that would cause the assertion to fail:

(A)	(B)	(C)	(D)	✓
(A)	(C)	(B)	(D)	✓
(A)	(C)	(D)	(B)	✓
(C)	(A)	(B)	(D)	✓
(C)	(A)	(D)	(B)	✓
(C)	(D)	(A)	(B)	✓

Proof by exhaustion (or full enumeration)!

A little combinatorial excursion

- Assuming 2 threads and k statements each, how many interleavings are there?
 - Any ideas?
- Hint 1
 - The merged list has length $k+k=2k$
 - Once we know which k positions in the merged list are occupied with elements from thread 1 (or 2) then the interleaving is determined!
 - How many are those?
- Hint 2
 - This is equivalent to sampling without replacement (draw the k positions out of 2k total)
“Ziehen ohne Zuruecklegen”
 - $\binom{2k}{k} = O\left(\frac{4^k}{\sqrt{2k}}\right)$
- If you cannot sleep tonight:
 - Generalize this to n threads 😊

Another proof

```

class C {
  private int x = 0;
  private int y = 0;
  Thread 1
    x = 1;
    y = 1;
  }
  Thread 2
    int a = y;
    int b = x;
    assert(b >= a);
  }
}
  
```

There is no interleaving of f and g causing the assertion to fail

Another proof (by contradiction):

Assume $b < a \Rightarrow a == 1$ and $b == 0$.

But if $a == 1 \Rightarrow y = 1$ *happened before* $a = y$.

And if $b == 0 \Rightarrow b = x$ *happened before* $x = 1$.

Because we assume that programs execute in order:

$a = y$ *happened before* $b = x$

$x = 1$ *happened before* $y = 1$

So by transitivity,

$a = y$ happened before $b = x$ happened before $x = 1$ happened before $y = 1$ happened before $a = y \Rightarrow$ **Contradiction** $\Leftarrow$

Let's try that on my laptop



Why it still can fail: Memory reordering

Rule of thumb: Compiler and hardware are allowed to make changes that do not affect the *semantics* of a *sequentially* executed program

```
void f() {  
    x = 1;  
    y = x+1;  
    z = x+1;  
}
```

semantically
equivalent?

```
void f() {  
    x = 1;  
    z = x+1;  
    y = x+1;  
}
```

semantically
equivalent?

```
void f() {  
    x = 1;  
    z = 2;  
    y = 2;  
}
```

In a **sequential** world!

Memory reordering: A software view

Modern compilers do not give guarantees that a global ordering of memory accesses is provided:

- Some memory accesses may be even optimized away completely!
- Class question: why?
- **Huge potential for optimizations – and for errors, when you make the wrong assumptions**
 - Dead code elimination
 - Register hoisting
 - Locality optimizations
 - ... many more (beyond this basic class, e.g., advanced dataflow – see DaCe)

Example: **Fail** with self-made rendezvous (C / GCC)

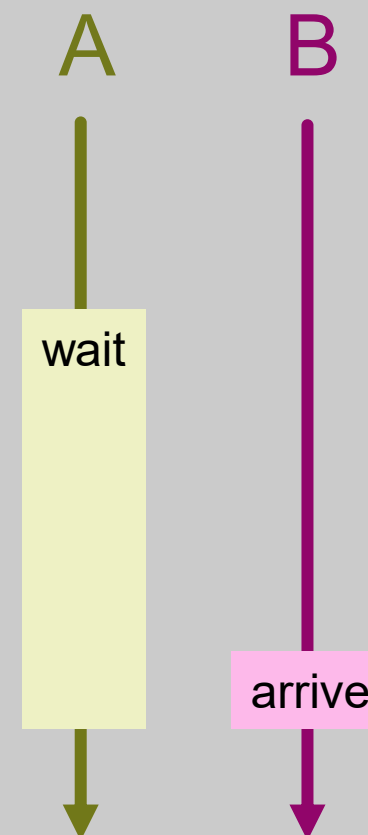
```
int x;

void wait() {
    x = 1;
    while(x==1);
}

void arrive() {
    x = 2;
}
```

Consider
thread A calls wait and
thread B subsequently calls arrive.

What would you naively expect?



Example: **Fail** with self-made rendezvous (C / GCC)

```
int x;

void wait() {
    x = 1;
    while(x==1);
}
```

```
void arrive() {
    x = 2;
}
```

Assembly without optimization

```
movl    $0x1, x
test:
mov     x, %eax
cmp     $0x1, %eax
je      test
```

je: jump (only) if equal,
i.e., if cmp yields true

```
movl    $0x2, x
```

Assembly with optimization

```
movl    $0x1, x
test:
jmp     test
```

jmp: jump always

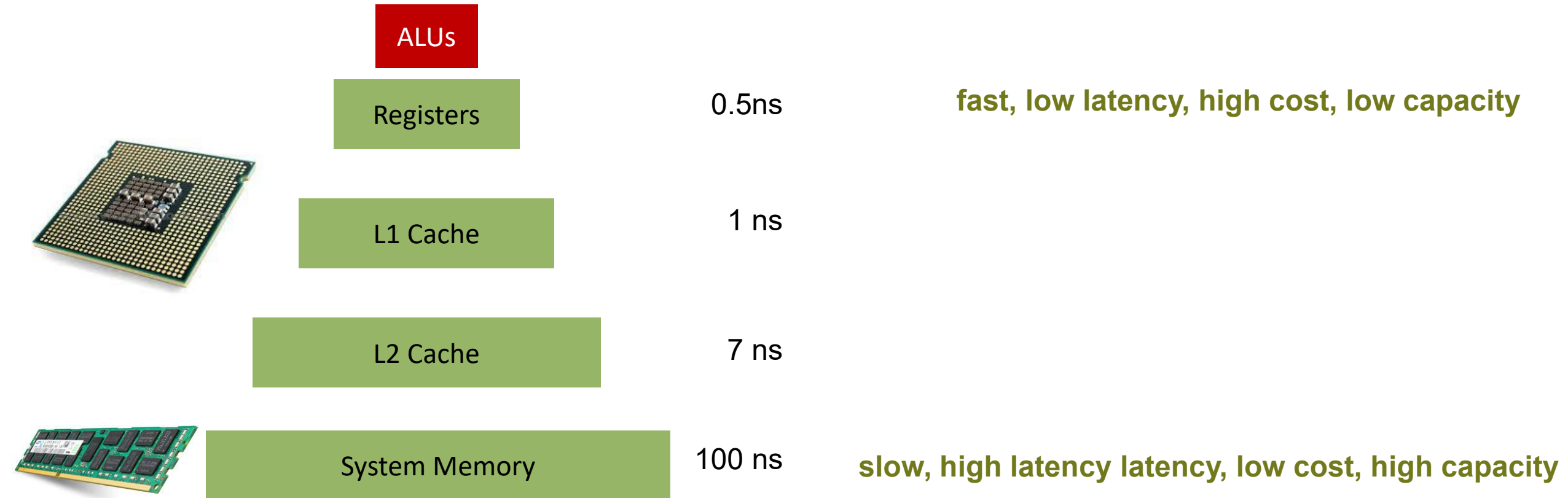
```
movl    $0x2, x
```

Memory reordering: A hardware view

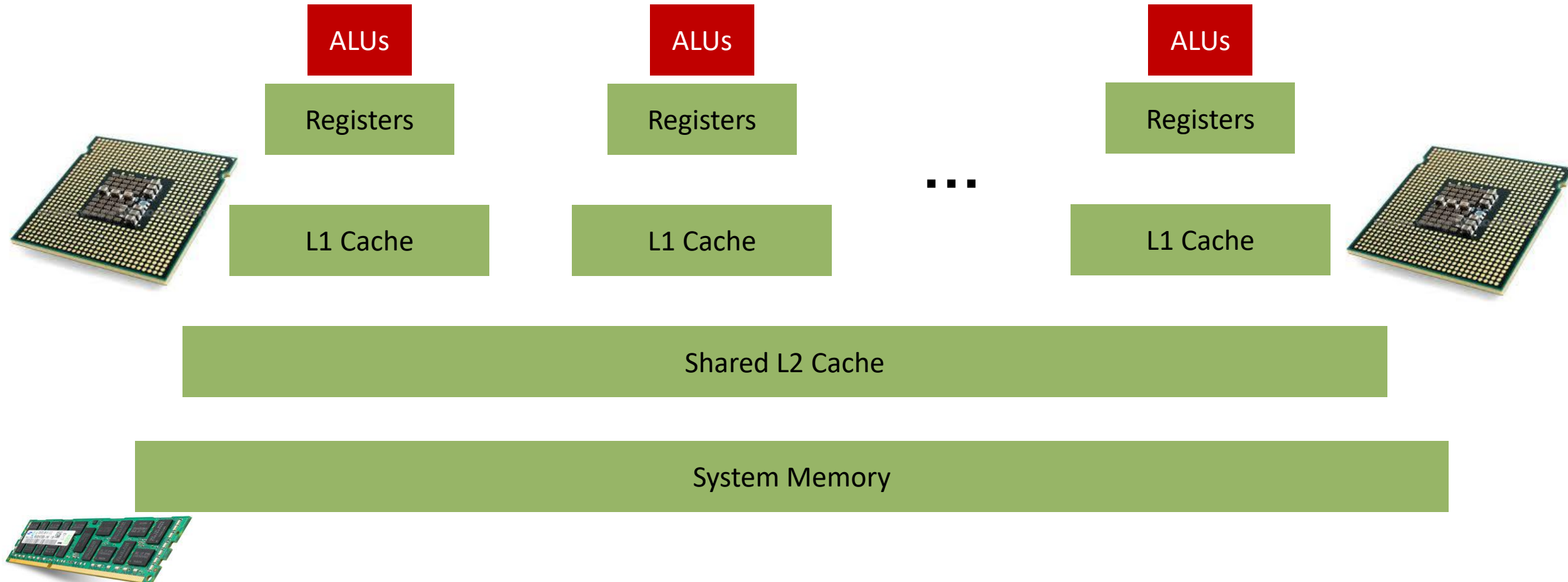
Modern multiprocessors do not enforce global ordering of all instructions:

- What they actually guarantee varies widely!
- Class question: why?
- **For performance!**
 - Most processors have a pipelined architecture and can execute (parts of) multiple instructions simultaneously. They can (and will) even reorder instructions internally.
 - Each processor has a local cache, and thus loads/stores to shared memory can become visible to other processors at different times

Memory hierachy (one core)



Memory hierachy (many cores)



A real-life analogy

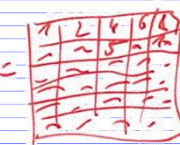
Anna

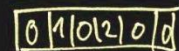
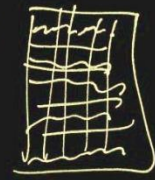
$C \leftarrow A \cdot C$
n times

$C =$  $A =$ 

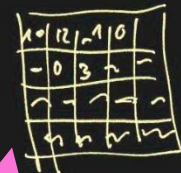
Beat

$z = V$
 $z = A^n \cdot z$

$z =$  $A =$ 

V  A  ϵ 

$h = 5$

B  z 

global data

Zoe

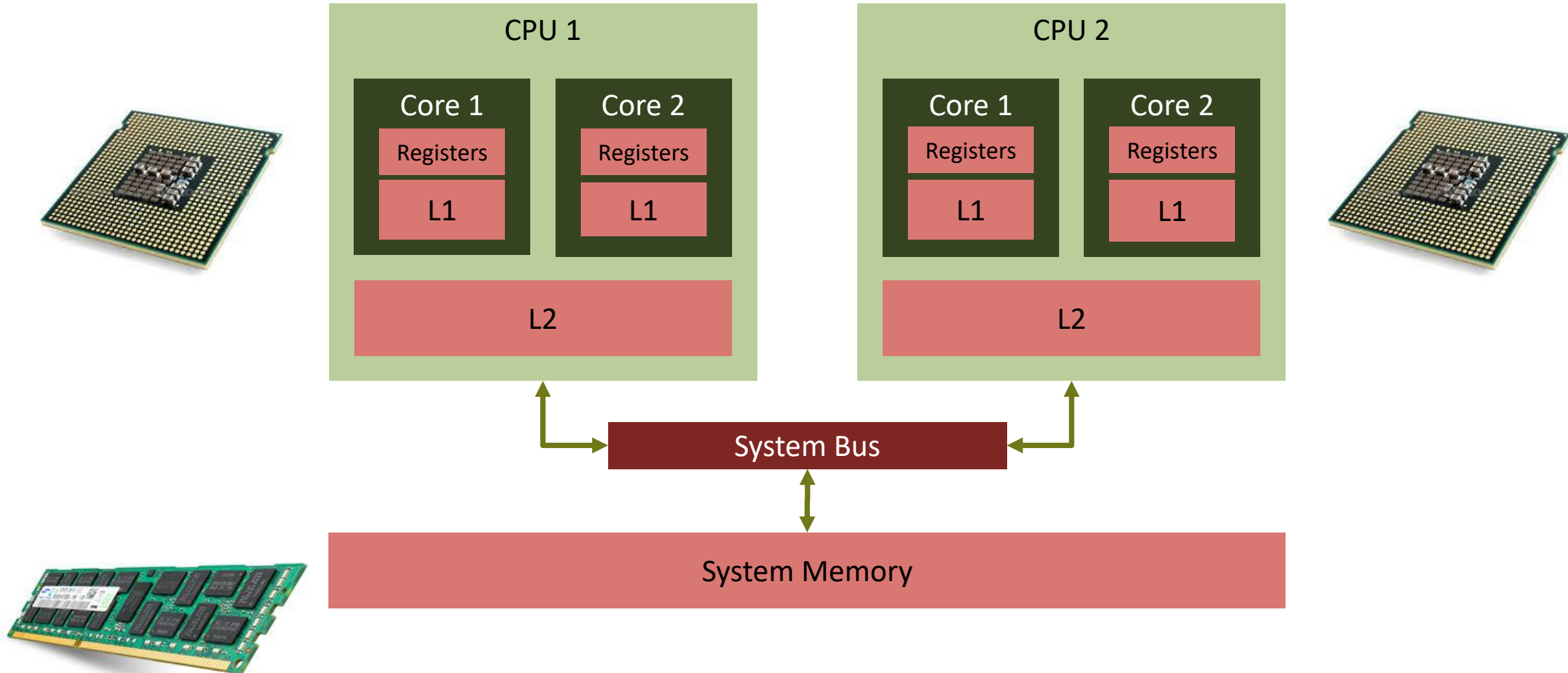
z 

Wait until $z \neq 0$

Then $V = B \cdot z$

local data

Sharing memory (schematically)



Memory models

The exact behavior of threads interacting via shared memory usually depends on hardware, runtime system, and programming language.

A memory model (e.g., of a programming language like Java) provides (often minimal) guarantees for the effects of memory operations.

- leaving optimization possibilities for hardware and compiler
- but including guidelines for writing correct multithreaded programs

Will come back to this later.

合同 (contract)

Implications

We need to learn (a bit) more about Java's Memory Model.

For now, we know that Java gives certain guarantees in the presence of synchronization.

Fixing our example

```
class C {  
    private int x = 0;  
    private int y = 0;  
    void f() {  
        synchronized(this) { x = 1; }  
        synchronized(this) { y = 1; }  
    }  
    void g() {  
        int a, b;  
        synchronized(this) { a = y; }  
        synchronized(this) { b = x; }  
        assert(b >= a);  
    }  
}
```

- Can use **synchronization** to avoid data races
- Then, indeed, the assertion cannot fail

Another fix

```
class C {  
    private volatile int x = 0;  
    private volatile int y = 0;  
    void f() {  
        x = 1;  
        y = 1;  
    }  
    void g() {  
        int a = y;  
        int b = x;  
        assert(b >= a);  
    }  
}
```

- Java has **volatile** fields: accesses do not count as data races
- Implementation: slower than regular fields, faster than locks
- Really for experts: avoid them if you can; use standard libraries instead
- And why do you need code like this anyway?



More realistic example of code that is wrong

```
class C {
    boolean stop = false;

    void f() {
        while(!stop) {
            // draw a monster
        }
    }

    void g() {
        stop = didUserQuit();
    }
}
```

Thread 1: $f()$

Thread 2: $g()$

No *guarantee* that Thread 1 will ever stop.

But honestly it will “*likely* work in practice”

What did we learn?

- **Compilers and computer architectures (processors)** will change orders of memory operations
 - Consistent with sequential semantics!
 - May impact parallel execution ☹️
- **There are some language constructs that forbid such reordering**
 - We saw synchronized and volatile in Java
 - But what do they really mean?
 - Now we need to dig a bit deeper (I'd rather not, but we have to)
It's quite complex!
- **Memory models**

RISC-V Memory Consistency Model Tutorial

Dan Lustig
May 7, 2018



WHY DO WE NEED A MEMORY MODEL?



...to give everyone a headache?

Why (architectural) memory models? For real ...

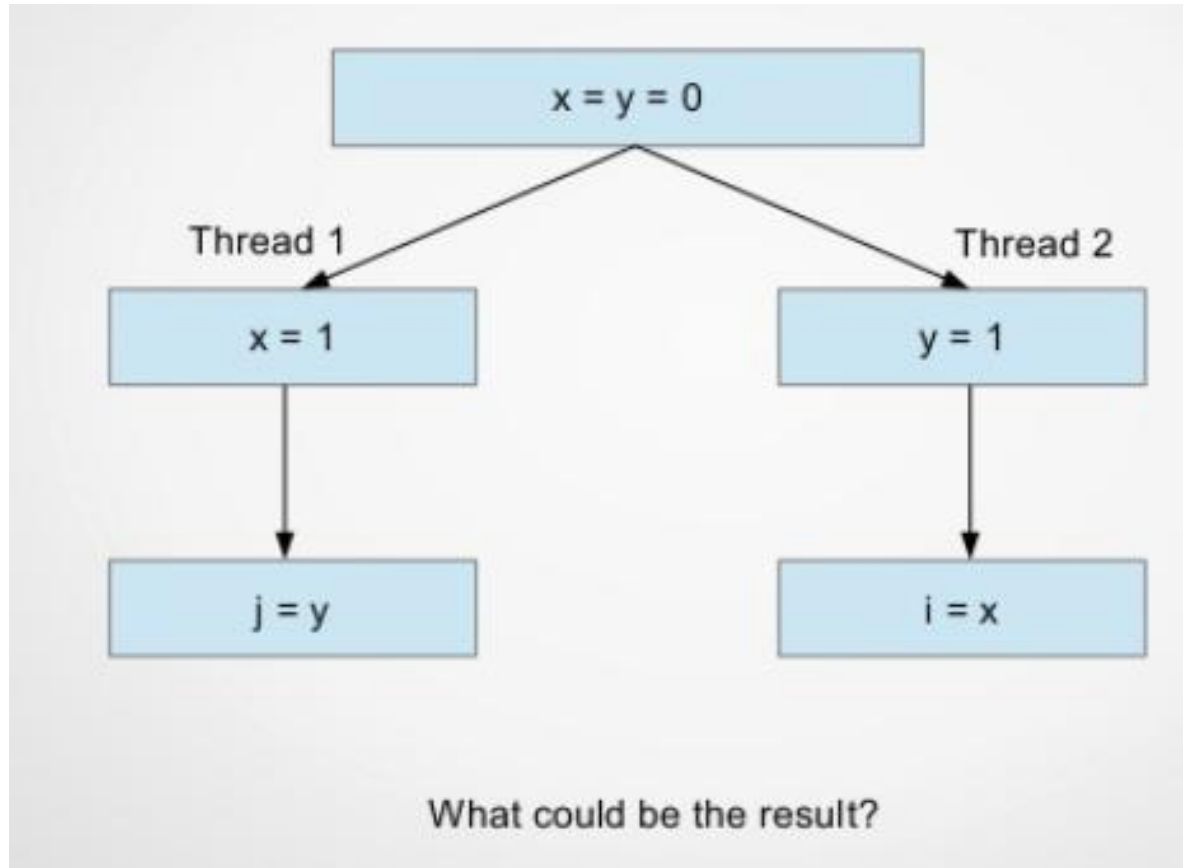
- You expect instructions to be executed in program order?
- But your compiler, your CPU, and your memory reorder! For better performance.
- What will be reordered depends on hardware, e.g., AMD64 is different than ARM.
 - In single threaded programs this does not cause problems.
- But we saw a shared-memory multithreading example using x86 fail us



Memory ordering in some architectures^{[7][8]}

Type	Alpha	ARMv7	PA-RISC	POWER	SPARC			x86		AMD64	IA-64	z/Architecture
					RMO	PSO	TSO		oostore ^[a]			
Loads reordered after loads	Y	Y	Y	Y	Y				Y		Y	
Loads reordered after stores	Y	Y	Y	Y	Y				Y		Y	
Stores reordered after stores	Y	Y	Y	Y	Y	Y			Y		Y	
Stores reordered after loads	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
Atomic reordered with loads	Y	Y		Y	Y						Y	
Atomic reordered with stores	Y	Y		Y	Y	Y					Y	
Dependent loads reordered	Y											
Incoherent instruction cache pipeline	Y	Y		Y	Y	Y	Y	Y	Y		Y	

Why memory models, x86 example



Answer:

i=1, j=1

i=0, j=1

i=1, j=0

i=0, j=0 (but why?)

Java Memory Model (JMM): Necessary basics

■ History ...

Java memory model

From Wikipedia, the free encyclopedia

The **Java memory model** describes how [threads](#) in the [Java programming language](#) interact through memory. Together with the description of single-threaded execution of code, the memory model provides the [semantics](#) of the Java programming language.

The original Java memory model, developed in 1995, was widely perceived as broken, preventing many runtime optimizations and not providing strong enough guarantees for code safety. It was updated through the [Java Community Process](#), as Java Specification Request 133 (JSR-133), which took effect in 2004, for [Tiger \(Java 5.0\)](#).^{[1][2]}

(To appear in Concurrency: Practice and Experience)

1995

The Java Memory Model is Fatally Flawed

William Pugh
 Dept. of Computer Science
 Univ. of Maryland, College Park
pugh@cs.umd.edu

FEATURE

Fixing the Java memory model



The Java Memory Model

2005

Jeremy Manson and William Pugh
 Department of Computer Science
 University of Maryland, College Park
 College Park, MD
{jmanson, pugh}@cs.umd.edu

Sarita V. Adve
 Department of Computer Science
 University of Illinois at Urbana-Champaign
 Urbana-Champaign, IL
sadve@cs.uiuc.edu

Close Encounters of The Java Memory Model Kind

2016

You Don't Get The Java Memory Model, Do You? But It Means Something. Something Important.



Is Java Memory Model still in use?

1 Answer

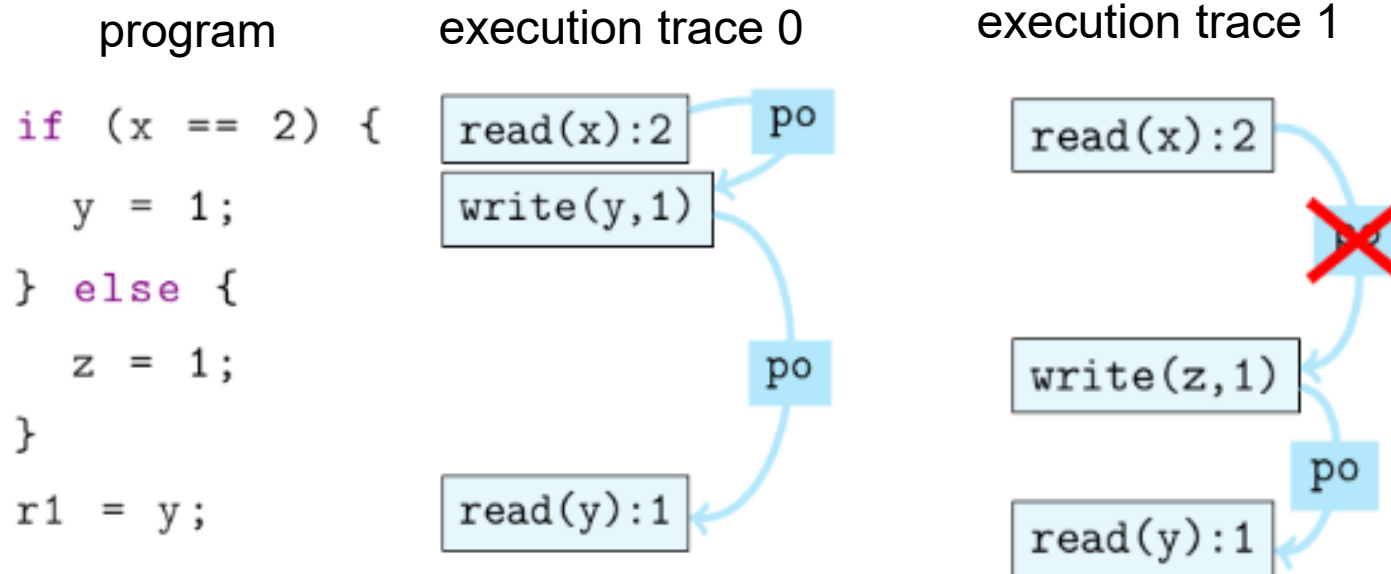
- ▲ So, is JMM still useful for modern Java?
- 1 Not really.
- ▼ So, is JMM still valid for modern Java?
- Yes, is still valid

Java Memory Model (JMM): Necessary basics

- **JMM restricts allowable outcomes of programs**
 - You saw that if we don't have these operations (volatile, synchronized etc.) – outcome can be “arbitrary” (not quite correct, say unexpected 😊)
- **JMM defines *Actions*: `read(x) : 1` “read variable x, the value read is 1”**
- ***Executions combine actions with ordering:***
 - *Program Order*
 - *Synchronizes-with*
 - *Synchronization Order*
 - *Happens-before*

JMM: Program Order (PO)

- **Program order is a total order of intra-thread actions – defined on an execution trace**
 - Program statements are NOT a total order **across** threads!
- **Program order does not provide an ordering guarantee for memory accesses!**
 - The only reason it exists is to provide the link between possible executions and the original program.
- **Intra-thread consistency: Per thread, the PO order is consistent with the thread's isolated execution**



JMM: Synchronization Actions (SA) and Synchronization Order (SO)

- **Synchronization Actions form the Synchronization Order (SO)**

- SO is a total order
- All threads see SA in the same order
- SA within a thread are in PO
- SO is consistent: all reads in SO see the last writes in SO

- **Synchronization actions are:**

- Read/write of a volatile variable
- Lock monitor, unlock monitor
- First/last action of a thread (synthetic)
- Actions which start a thread
- Actions which determine if a thread has terminated

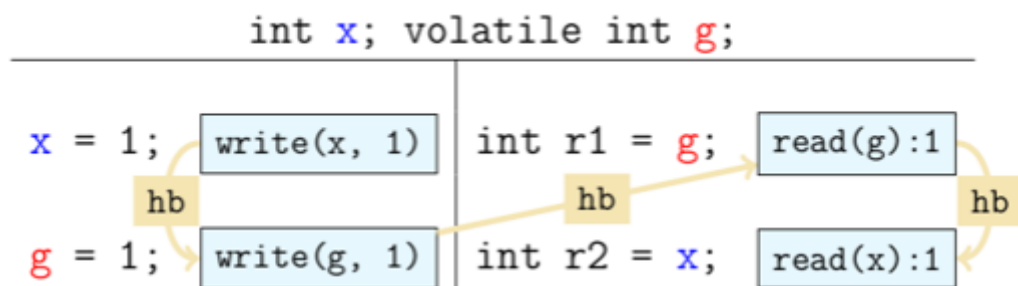
volatile int x , y ;	
x = 1;	y = 1;
int r1 = y ;	int r2 = x ;

Exercise: List all outcomes (r1,r2) allowed by the JMM.

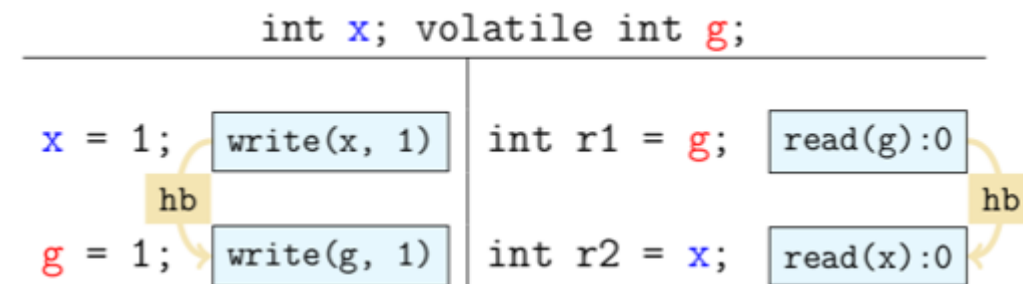
JMM: Synchronizes-With (SW) / Happens-Before (HB) orders

- Synchronizes with (SW) only pairs the specific actions which "see" each other
- A volatile write to x synchronizes with subsequent read of x (subsequent in SO)
- The transitive closure of PO and SW forms HB
- HB consistency: When reading a variable, we see either the last write (in HB) or any other unordered write.
 - This means races are allowed in Java!

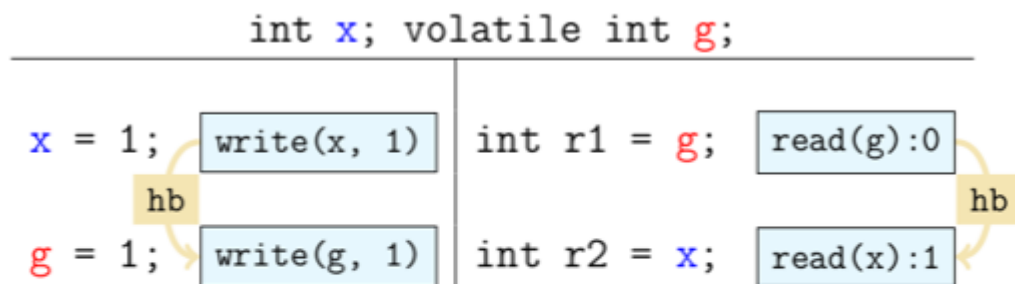
Example



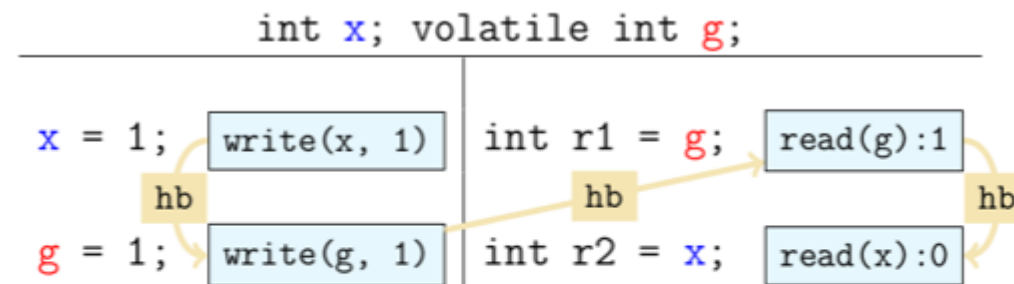
Case 1: HB consistent, observe the latest write in $\xrightarrow{\text{hb}}$
 $(r1, r2) = (1, 1)$



Case 2: HB consistent, observe the default value
 $(r1, r2) = (0, 0)$



Case 3: HB consistent (!), reading via race!
 $(r1, r2) = (0, 1)$



Case 4: HB **inconsistent**, execution can be thrown away

Behind Locks

Implementation of Mutual Exclusion

Assumptions

In the following we assume

We will make «atomic» more precise later.

- 1) atomic reads and writes of variables of primitive type
- 2) no reordering of read and write sequences (! not true in practice ! here for simplicity !)
- 3) threads calling lock (enter a critical section) will eventually call (and complete) unlock

Otherwise we assume a multithreaded environment where processes can arbitrarily interleave.

We make no assumptions for progress in non-critical section!

Critical sections (e.g., locks)

Pieces of code with the following conditions

1. **Mutual exclusion:** statements from critical sections of two or more processes must not be interleaved
2. **Freedom from deadlock:** if some processes are trying to enter a critical section then one of them must eventually succeed
3. **Freedom from starvation:** if *any* process tries to enter its critical section, then that process must eventually succeed

Critical section problem

global (shared) variables

Process P

local variables

loop

non-critical section

preprotocol

critical section

postprotocol

Process Q

local variables

loop

non-critical section

preprotocol

critical section

postprotocol

Easy to implement on a
single-core machine.
How?

Easy to implement on a single core system ...

global (shared) variables

Process P

local variables

loop

non-critical section

Switch off IRQs

critical section

Switch on IRQs

Process Q

local variables

loop

non-critical section

Switch off IRQs

critical section

Switch on IRQs

Mutual exclusion for 2 processes -- 1st Try

```
volatile boolean wantp=false, wantq=false
```

Process P

local variables

loop

p1 non-critical section

p2 while(wantq);

p3 wantp = true

p4 critical section

p5 wantp = false

Process Q

local variables

loop

q1 non-critical section

q2 while(wantp);

q3 wantq = true

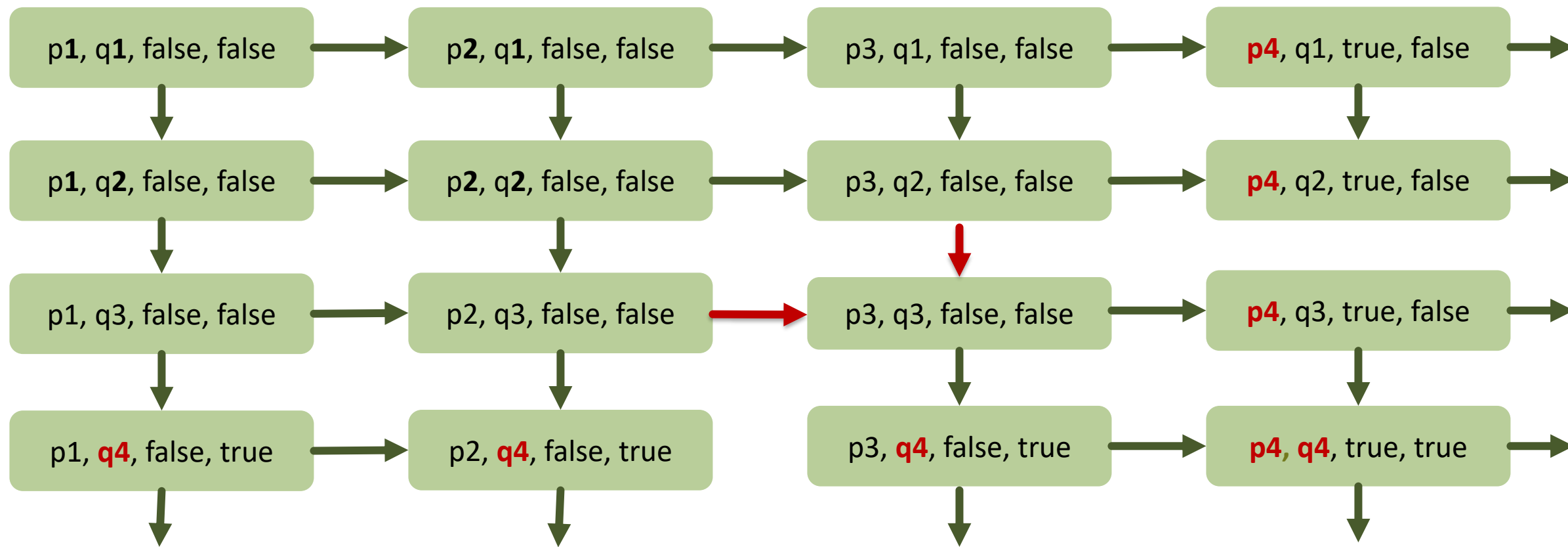
q4 critical section

q5 wantq = false

Do you see the problem?

State space diagram [p, q, wantp, wantq]

- 1 non-critical section 2 while(wantp) while(wantq) 3 wantp = true wantq = true 4 **critical section** 5 wantp = false wantq = false



no mutual exclusion !

Observation: state space diagram too large

volatile bool

Process P

local variables

loop

p1 non-critical section

p2 while(wantq);

p3 wantp = true

p4 critical section

p5 wantp = false

Only of interest: state transitions of the protocol.

p1/q1 is identical to p2/q2 – call state 2

p4/q4 is identical to p5/q5 – call state 5

Then forbidden: both processes in state 5

loop

q1 non-critical section

q2 while(wantp);

q3 wantq = true

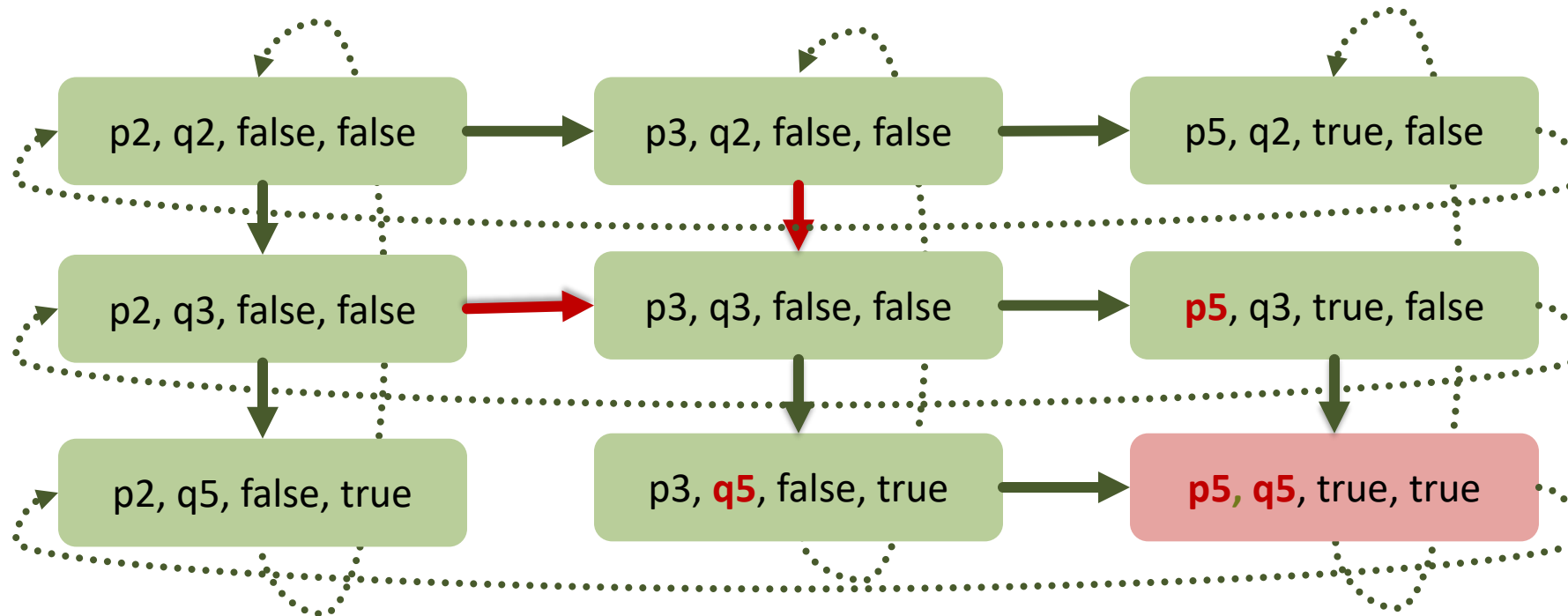
q4 critical section

q5 wantq = false

Reduced state space diagram [p, q, wantp, wantq] – only states 2, 3, and 5

1 non-critical section → 2 await wantq == false
 await wantp == false → 3 wantp = true
 wantq = true → 4 critical section → 5 wantp = false
 wantq = false

All of interest covered:



no mutual exclusion !

Mutual exclusion for 2 processes -- 2nd Try

volatile boolean wantp=false, wantq=false

Process P

local variables

loop

p1 non-critical section

p2 wantp = true

p3 while(wantq);

p4 critical section

p5 wantp = false

Process Q

local variables

loop

q1 non-critical section

q2 wantq = true

q3 while(wantp):

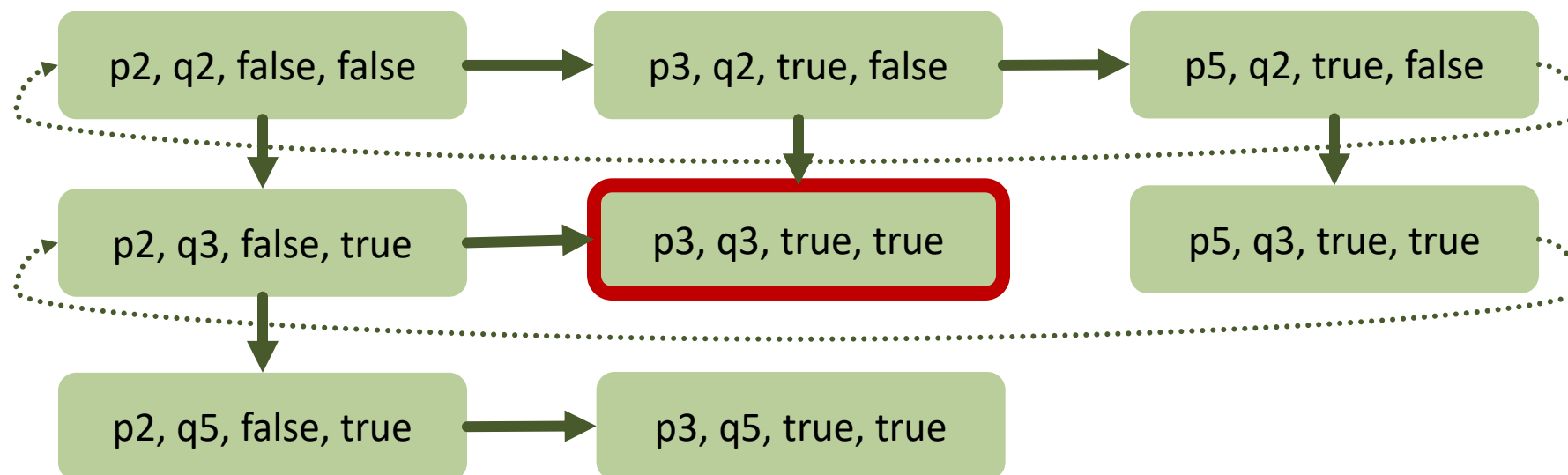
q4 critical section

q5 wantq = false

Do you see the problem?

State space diagram [p, q, wantp, wantq]

- 1 non-critical section 2 wantp = true
 wantq = true 3 while(wantp)
 while(wantq) 4 critical section 5 wantp = false
 wantq = false



deadlock !

Mutual exclusion for 2 processes -- 3rd Try

```
volatile int turn = 1;
```

Process P

local variables

loop

p1 non-critical section

p2 while(turn != 1);

p3 **critical section**

p4 turn = 2

Process Q

local variables

loop

q1 non-critical section

q2 while(turn != 2);

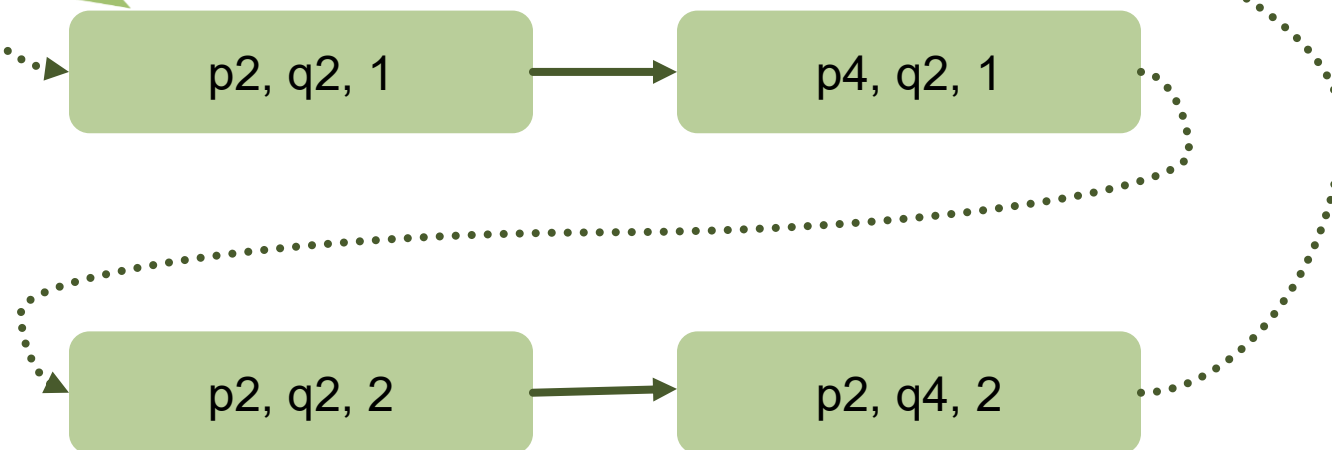
q3 **critical section**

q4 turn = 1

Do you see the problem?

State space diagram [p, q, turn]

We have not made any assumptions about progress outside of the CS...



starvation!

Process P

local variables

loop

p1 non-critical section

p2 while(turn != 1);

p3 critical section

p4 turn = 2

Process Q

local variables

loop

q1 non-critical section

q2 while(turn != 2);

q3 critical section

q4 turn = 1

A combination of the tries 2 and 3: Decker's Algorithm

volatile boolean wantp=false, wantq=false, integer turn= 1

Process P

loop

non-critical section

wantp = true

while (wantq) {

if (turn == 2) {

wantp = false;

while(turn != 1);

wantp = true; }}

critical section

turn = 2

wantp = false

only when q
tries to get
lock

and q has
preference

let q proceed

and wait

and try again

Process Q

loop

non-critical section

wantq = true

while (wantp) {

if (turn == 1) {

wantq = false

while(turn != 2);

wantq = true; }}

critical section

turn = 1

wantq = false

More concise than Decker: Peterson Lock

```
let P=1, Q=2; volatile boolean array flag[1..2] = [false, false];
volatile integer victim = 1
```

Process P (1)

loop

non-critical section

flag[P] = true

victim = P

while(flag[Q] && victim == P);

critical section

flag[P] = false

I am
interested

but you go
first

We both are
interested

And you go first

Process Q (2)

loop

non-critical section

flag[Q] = true

victim = Q

while(flag[P] && victim == Q);

critical section

flag[Q] = false

We want to prove ...

**that the Peterson Lock satisfies mutual exclusion
and that it is starvation free**

How?

Requires some notation first.

Events and precedence

Threads produce a sequence of events

P produces events $p_0, p_1, \dots$

e.g., $p_1 = \text{"flag[P] = true"}$

j-th occurrence of event i in thread P: p_i^j

e.g., $p_5^3 = \text{"flag[P] = false"}$ in the third iteration

Precedence relation: we write $a \rightarrow b$ when a occurs before b.

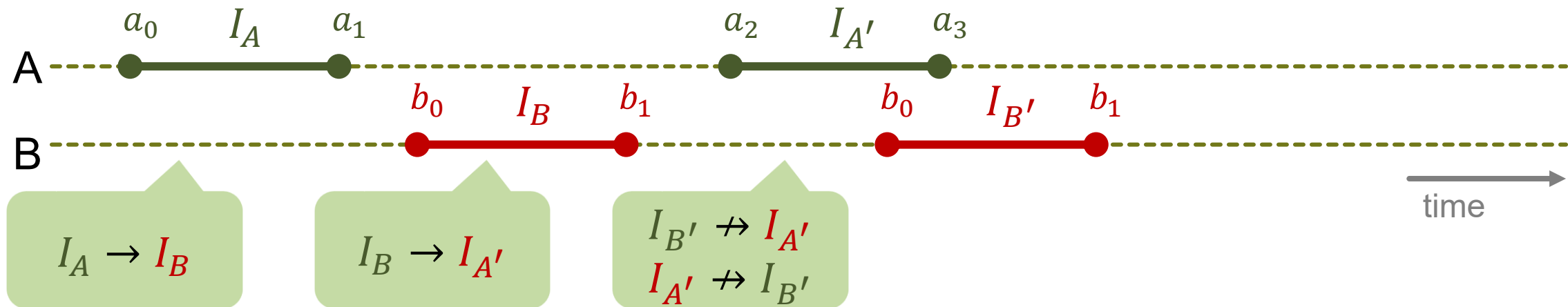
Note that the precedence relation " $\rightarrow$ " is a total order for events.

programs usually consist of loops,
therefore we might need to count
occurrences

Intervals

(a_0, a_1) : interval of events a_0, a_1 with $a_0 \rightarrow a_1$

With $I_A = (a_0, a_1)$ and $I_B = (b_0, b_1)$ we write $I_A \rightarrow I_B$ if $a_1 \rightarrow b_0$



we say " I_A precedes I_B " and " $I_{B'}$ and $I_{A'}$ are **concurrent**"

Atomic register

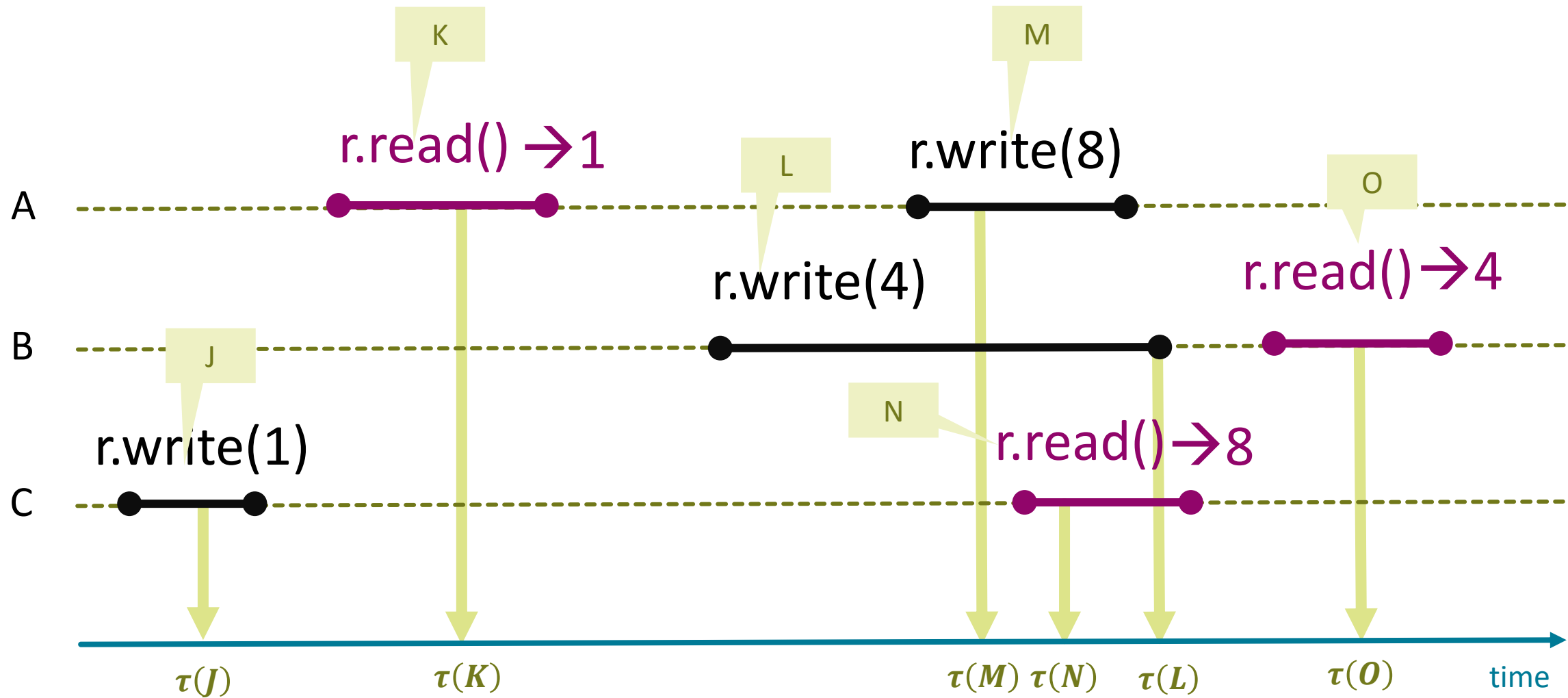
Register: basic memory object, can be shared or not
 i.e., in this context register $\neq$ register of a CPU

Register r : operations $r.read()$ and $r.write(v)$

Atomic Register:

- An invocation J of $r.read$ or $r.write$ takes effect at a single point $\tau(J)$ in time
- $\tau(J)$ always lies between start and end of the operation J
- Two operations J and K on the same register always have a different effect time $\tau(J) \neq \tau(K)$
- An invocation J of $r.read()$ returns the value v written by the invocation K of $r.write(v)$ with closest preceding effect time $\tau(K)$

Example



Atomic register

Assumptions for Atomic Registers justify to treat operations on them as events taking place at a single point in time.

Will use this in the following proofs.

Note that even with atomic registers there can still be non-determinism of programs because nothing is said about the order of effect times for concurrent operations.

Proof: Mutual exclusion (Peterson)

By contradiction: assume concurrent CS_P and CS_Q [A]

Assume without loss of generality:

```
flag[P] = true
victim = P
while (flag[Q] && victim == P){}
CSP
flag[P] = false
```

$$W_Q(\text{victim}=Q) \rightarrow W_P(\text{victim}=P) \text{ [B]}$$

From the code:

$$W_P(\text{flag}[P]=\text{true}) \rightarrow W_P(\text{victim} = P) \rightarrow R_P(\text{flag}[Q]) \rightarrow R_P(\text{victim}) \rightarrow CS_P$$

"write of P"

A + C $\Rightarrow$ must read false

B $\Rightarrow$ must read P [C]

transitivity of " $\rightarrow$ "
 $\Rightarrow$ must read true



$$W_Q(\text{flag}[Q]=\text{true}) \rightarrow W_Q(\text{victim} = Q) \rightarrow R_Q(\text{flag}[P]) \rightarrow R_Q(\text{victim}) \rightarrow CS_Q$$

"read of Q"

Proof: Freedom from starvation

```
flag[P] = true
victim = P
while (flag[Q] && victim == P){}
CSp
flag[P] = false
```

By (exhaustive) contradiction

Assume without loss of generality that P runs forever in its lock loop, waiting until `flag[Q]==false` or `victim != P`.

Possibilities for Q:

stuck in nonCS

⇒ `flag[Q] = false` and P can continue. Contradiction.

repeatedly entering and leaving its CS

⇒ sets `victim` to Q when entering.

Now `victim` cannot be changed ⇒ P can continue. Contradiction.

stuck in its lock loop waiting until `flag[P]==false` or `victim != Q`.

But `victim == P` and `victim == Q` cannot hold at the same time. Contradiction.

Peterson in Java

```
class PetersonLock
{
    volatile boolean flag[] = new boolean[2];
    volatile int victim;

    public void Acquire(int id)
    {
        flag[id] = true;
        victim = id;
        while (flag[1-id] && victim == id);
    }

    public void Release(int id)
    {
        flag[id] = false;
    }
}
```

Volatile reference to an array and not an array of volatile variables!
 This example may work in practice. However, for production programs one must use Java's `AtomicInteger` and `AtomicIntegerArray`.

More than two threads

- How to extend Peterson's lock to more than 2 threads?
- Think about it, I will present a solution in tomorrow's lecture.